

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – FLOWCHART-TO-CODE CONVERSION

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO5 – Naturalization (BL4, BL6)	Assessment:	Assessment Tool 2 – Flowchart-To-Code Conversion
Weightage:	35% of CIA	Total Marks:	100
CO5 Statement:	Construct MST using shortest path algorithms and traverse the graph to model and analyse real-world problem.		
Student Name:	N. Sathwik	Reg. No.:	192525077
Section / Batch:	2/2025	Date:	02/09/2026

Code of Conduct

I N. Sathwik (192525077) (Name / Reg No)
certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the Student: N. Sathwik

Instructions

- This test contains 5 Flowchart-to-code conversion questions. Total: 100 Marks.
- When a student answer using Flowchart-to-code conversion should evaluate based on the ability to design clear, logical, and efficient code solutions for data structure problems.
- No negative marking. Unanswered questions carry zero marks.

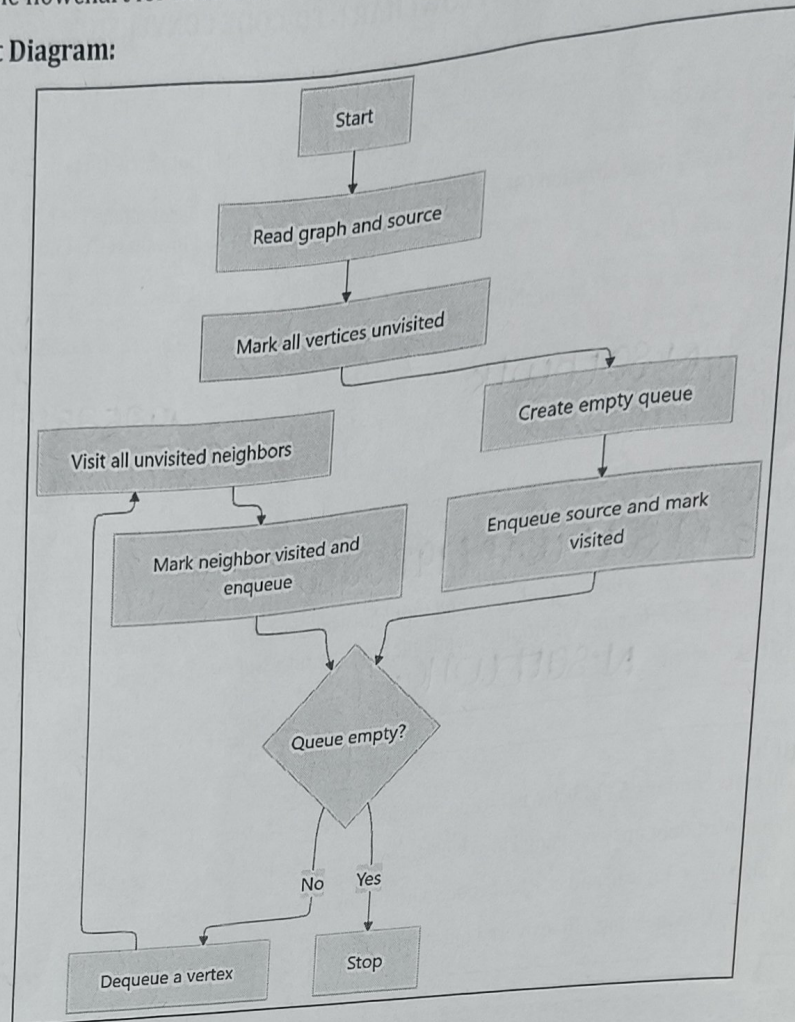
Section / Topic	Focus	Q Nos	Marks
Graph Traversal	BFS	1	20
Minimum Spanning Tree	Kruskal's Algorithm	2	20
Graph Sorting	Topological Sort	3	20
Graph Traversal	DFS	4	20
Minimum Spanning Tree	Prim's Algorithm	5	20
GRAND TOTAL:			100 Marks

Annexure A - Flowchart-To-Code Conversion

(20 Marks)

1. Convert the flowchart for Breadth First Search traversal into code.

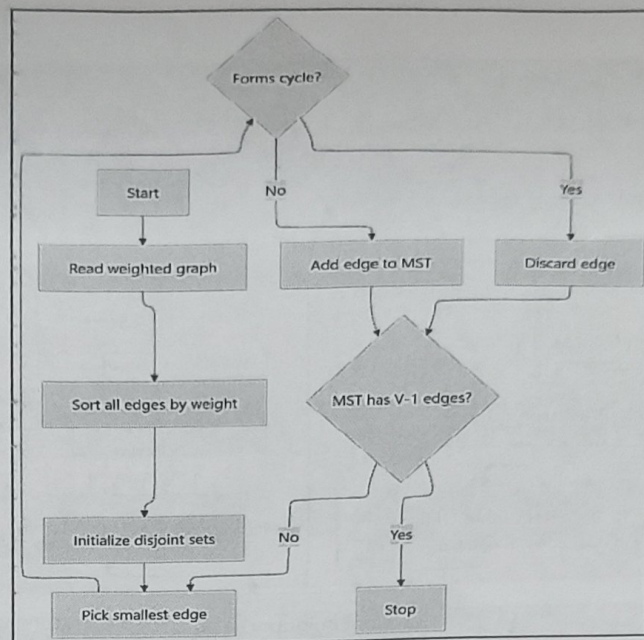
Flowchart Diagram:



2. Convert the flowchart for Kruskal's algorithm into code.

(20 Marks)

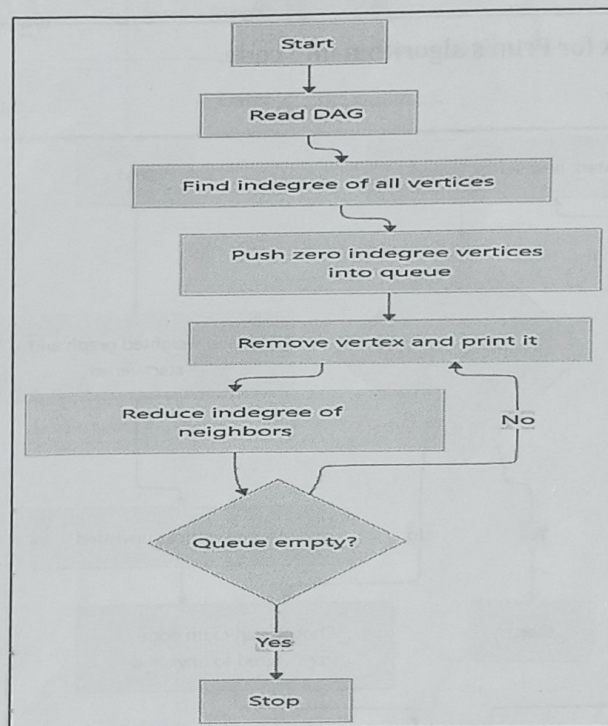
Flowchart Diagram:



3. Convert the flowchart for topological sort into code.

(20 Marks)

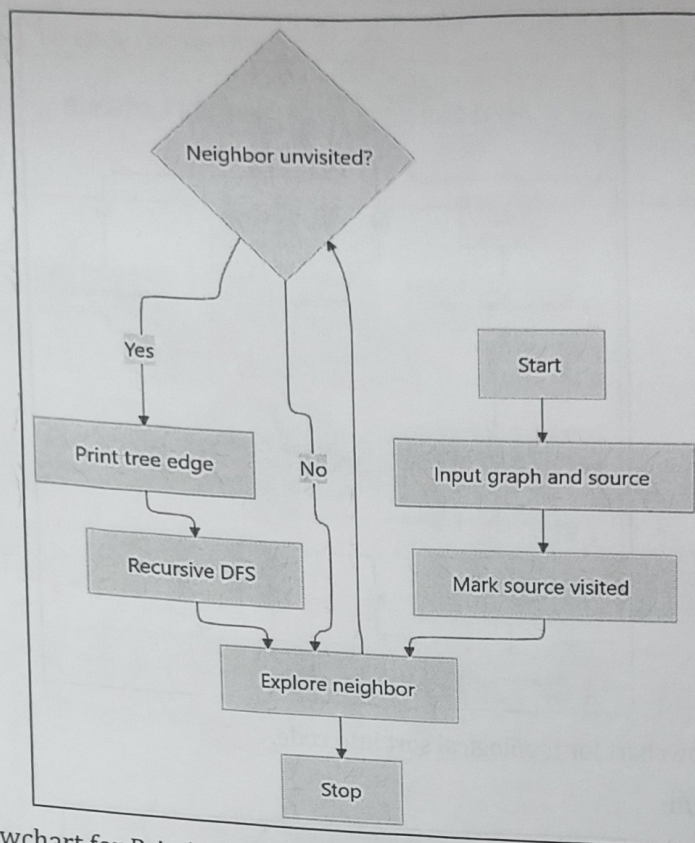
Flowchart Diagram:



4. Convert the flowchart for generating a DFS tree into code.

(20 Marks)

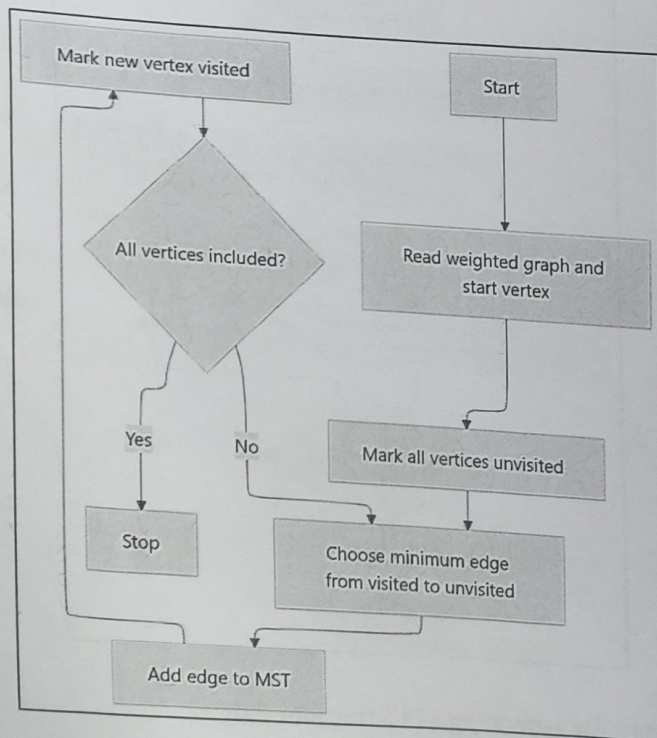
Flowchart Diagram:



5. Convert the flowchart for Prim's algorithm into code.

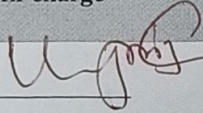
(20 Marks)

Flowchart Diagram:



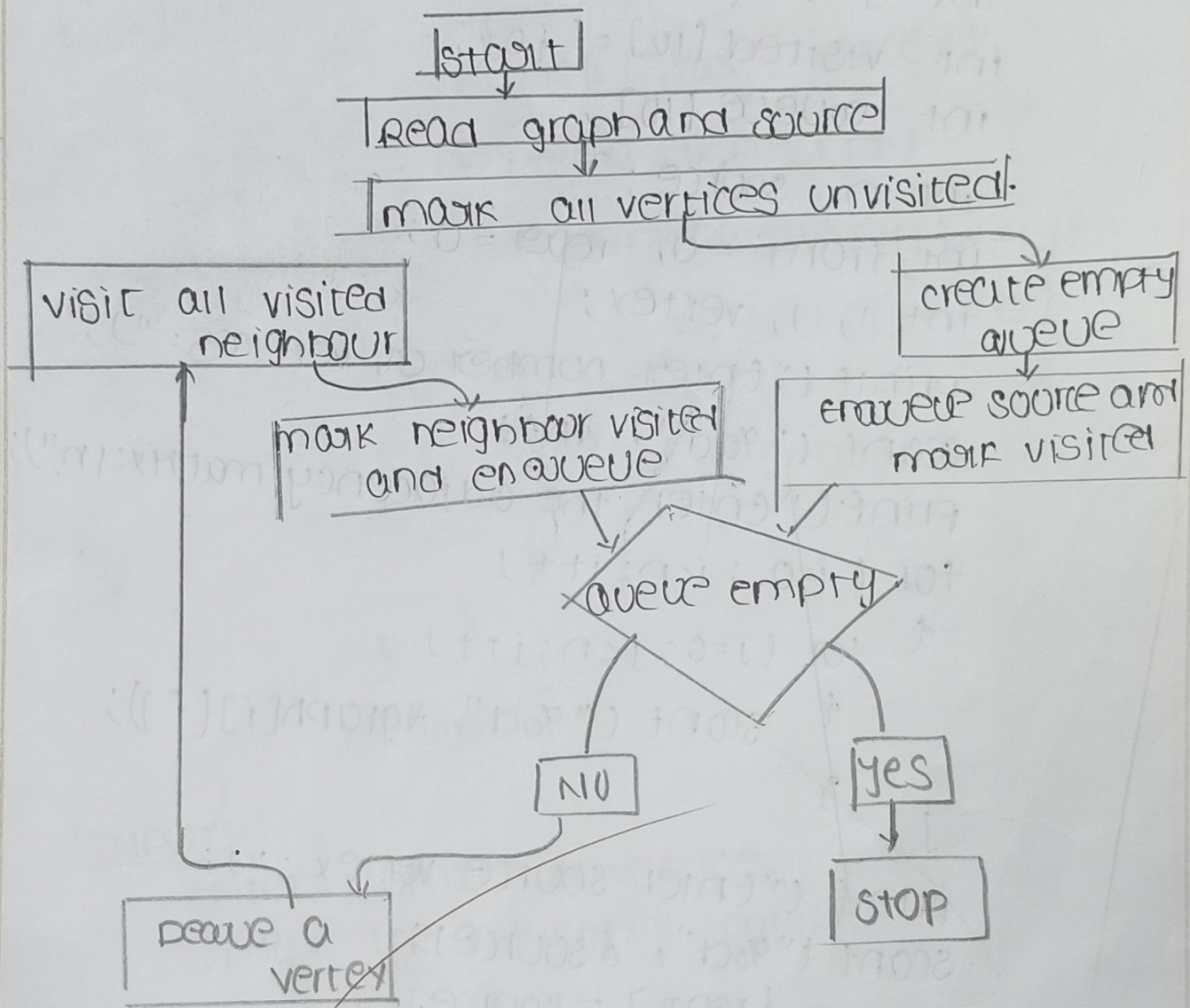
Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20	Demonstrates complete and clear understanding; handles edge cases effectively	Good understanding; minor gaps in edge case handling	Partial understanding; misses some requirements	Misinterprets problem; major gaps
Code Correctness	30	Fully correct; passes all test cases	Mostly correct; minor errors	Partially correct; several test cases fail	Incorrect or non-functional code
Code Quality & Readability	20	Clean, modular, well-commented, follows standards	Readable with some structure	Limited readability; poor structure	Unorganized and hard to read
Complexity Analysis	20	Accurate time & space complexity with justification	Correct but limited explanation	Incomplete or partially correct	Missing or incorrect
Testing & Validation	10	Thorough testing including edge cases	Adequate testing with few edge cases	Minimal testing	No testing evidence

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1. convert the flow chart for Breadth First search traversal into code.

Flowchart diagram:



Ans

```
#include <stdio.h>

int main()
{
    int graph[10][10];
    int visited[10] = {0};
    int queue[10];

    int n, source;
    int front = 0, rear = 0;
    int i, j, vertex;

    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter the adjacency matrix:\n");
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            scanf("%d", &graph[i][j]);
        }
    }

    printf("Enter source vertex: ");
    scanf("%d", &source);
    queue[rear] = source;
    rear++;
    visited[source] = 1;
```



```
printf (" BFS traversal:");
```

```
while (front < rear)
```

```
{ vertex = dequeue (front);
```

```
front++;
```

```
printf ("%d", vertex);
```

```
for (i=0; i<n; i++)
```

```
{ if (graph[vertex][i] == 1 &&
```

```
visited[i] == 0)
```

```
visited[i] = 1;
```

```
enqueue [rear] = i;
```

```
rear;
```

```
}
```

```
}
```

```
}
```

```
return 0
```

INPUT:-

Enter number of vertices : 5

Enter the adjacency matrix:

0 1 1 0 0

1 0 0 1 0

1 0 0 1 1

0 1 1 0 1

0 0 1 1 0

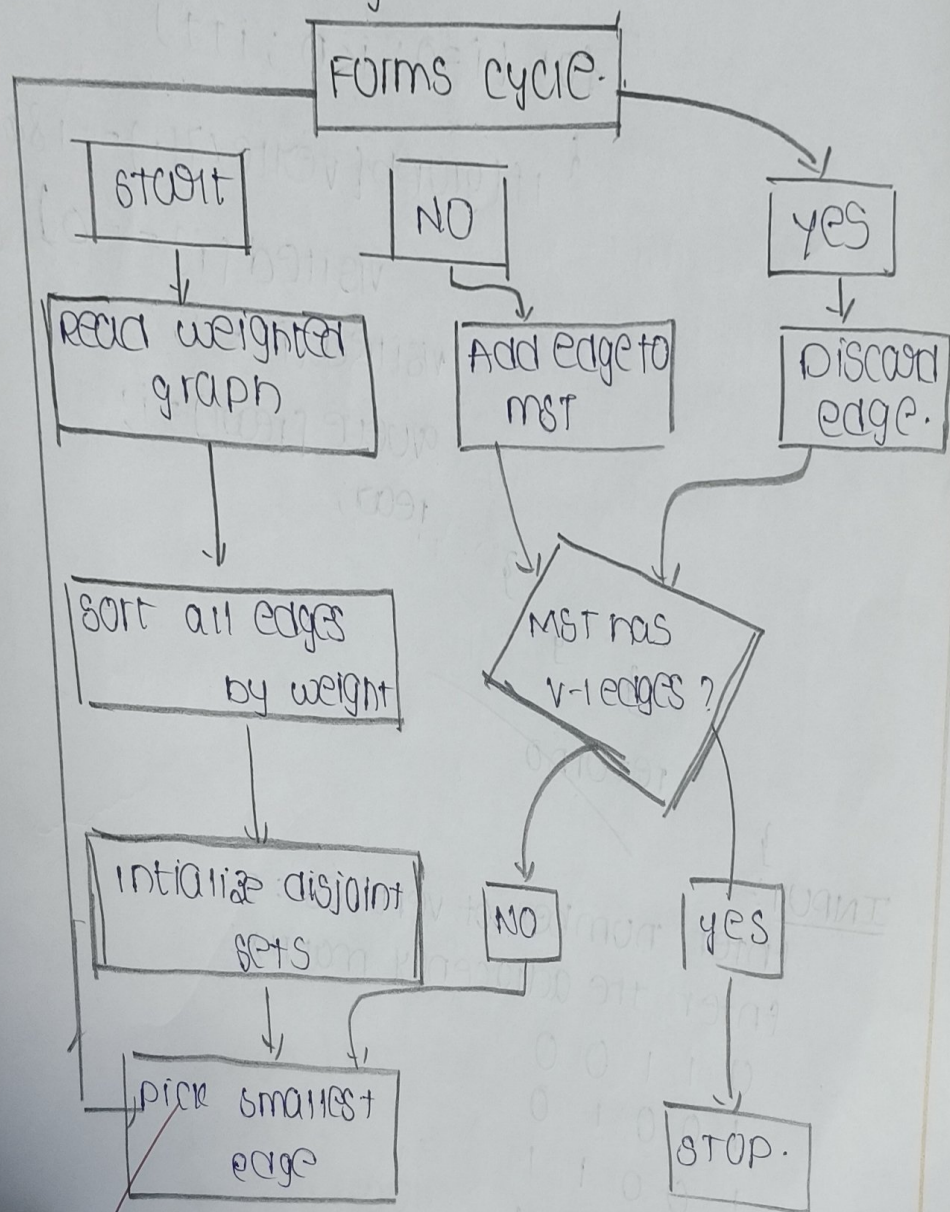
Enter source vertex : 0.

Output:

BFS Traversal:- 0 1 2 3 4.

(b) Convert the flowchart for KRUSKAL'S algorithm into code

Flowchart diagram:



Ans

```
#include <stdio.h>
```

```
struct edge
```

```
{  
    int u, v, weight;
```

```
};
```

```
int parent[20];
```

```
int find (int i)
```

```
{  
    while (parent[i] != i)
```

```
        i = parent[i];
```

```
    return i;
```

```
}
```

```
void unionset (int u, int v)
```

```
{  
    int rootu = find(u);
```

```
    int rootv = find(v);
```

```
    parent[rootv] = rootu;
```

```
}
```

```
int main()
```

```
{  
    struct edge edges[50], temp;
```

```
    int v, e;
```

```
    int i, j;
```

```
    int count = 0;
```

```
    int totalcost = 0;
```



```
printf("enter number of vertices:");
```

```
scanf("%d", &v);
```

```
printf("enter number of edges:");
```

```
scanf("%d", &e);
```

```
printf("enter edges (source destination  
weight):");
```

```
for (i=0; i<e; i++)
```

```
{  
    scanf("%d %d %d",  
        &edges[i].u,  
        &edges[i].v,  
        &edges[i].weight);  
}
```

```
for (i=0; i<e-1; i++)
```

```
{  
    for (j=0; j<e-i-1; j++)
```

```
{  
    if (edges[j].weight > edges[j+1].weight)
```

```
{  
        temp = edges[j];
```

```
        edges[j] = edges[j+1];
```

```
        edges[j+1] = temp;
```

```
    }
```

```
}
```

```
}
```



```
for (i=0; i<v; i++)
```

```
{ parent[i] = i;
```

```
}
```

```
printf ("Edges in minimum spanning  
tree: \n");
```

```
for (i=0; i<E && count < v-1; i++)
```

```
{ int u = edges[i].u;
```

```
int v = edges[i].v;
```

```
if (find(u) != find(v))
```

```
printf ("%d - %d = %d \n",
```

```
u, v, edges[i].weight);
```

```
total cost = total cost + edges[i].weight;
```

```
unionset(u, v);
```

```
count++;
```

```
}
```

```
else
```

```
{ printf ("%d - %d = %d discarded.
```

```
(cycle) \n", u, v, edges[i].weight);
```

```
}
```

```
printf ("\n minimum cost = %d \n", totalcost);
```

```
return 0;
```

```
}
```


INPUT:-

enter number of vertices : 4

enter number of edges : 5

enter edges (source destination weight):

0 1 10

0 2 6

0 3 5

1 3 15

2 3 4

OUTPUT:

Edges in minimum spanning tree.

~~2 - - 3 = 4~~

~~0 - - 3 = 5~~

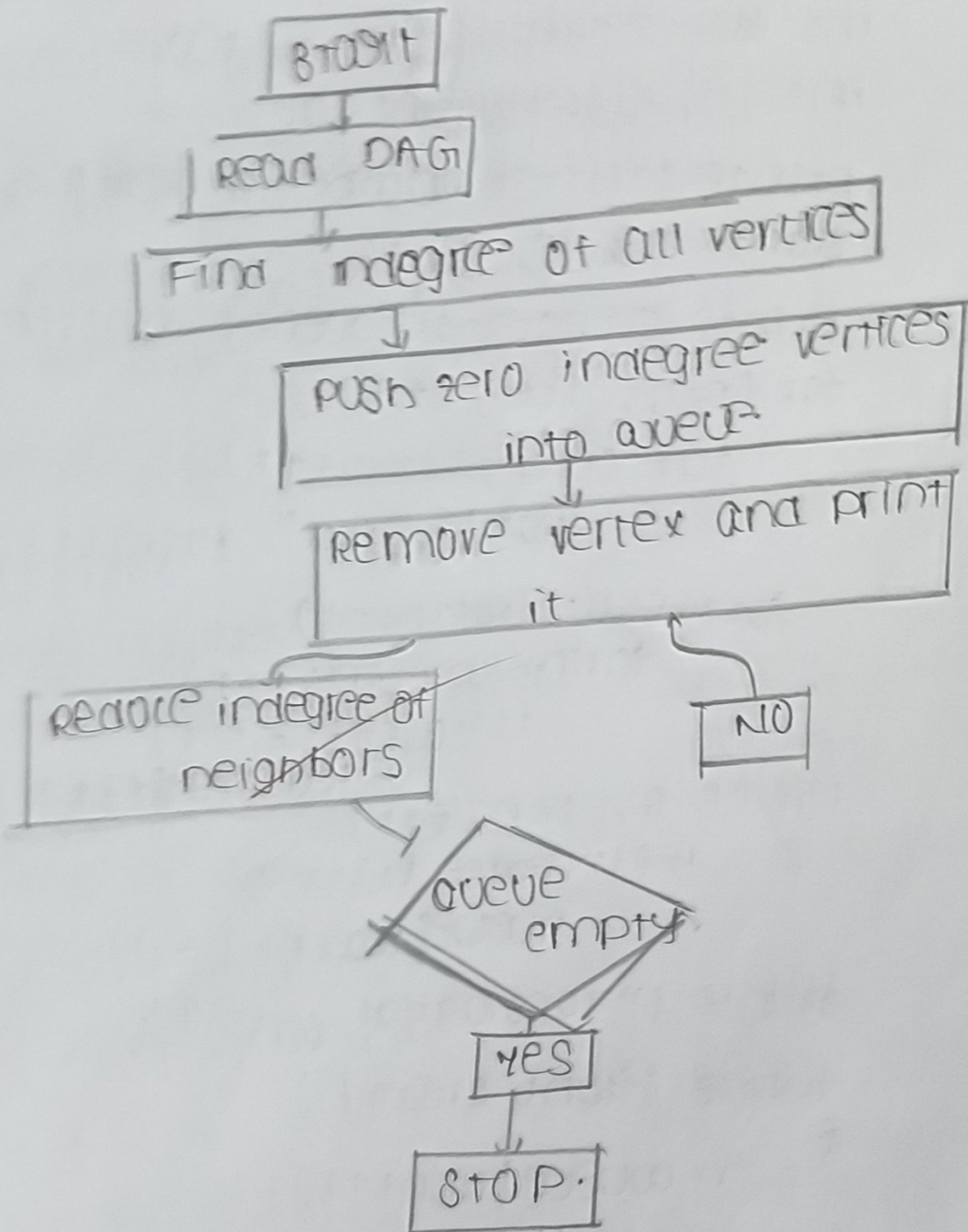
~~0 - - 2 = 6~~ discarded (cycle).

~~0 - - 1 = 10~~

minimum cost = 19.

3. Convert the flowchart for topological sort into code.

Ans. Flowchart diagram.



(A) #include <stdio.h>

int main()

{ int a[10][10], indegree[10] = {0};

int queue[10], front=0, rear=0;

int n, i, j, v, count=0;

printf ("Enter number of vertices : ");

scanf ("%d", &n);

printf ("Enter adjacency matrix:\n");

for (i=0; i<n; i++)

for (j=0; j<n; j++)

scanf ("%d", &a[i][j]);

for (i=0; i<n; i++)

for (j=0; j<n; j++)

if (a[i][j] == 1)

indegree[j]++;

for (i=0; i<n; i++)

if (indegree[i] == 0)

queue[rear++] = i;

printf ("Topological sort:");

while (front < rear)

{ v = queue[front++];

printf ("%d", v);

count++;


```

for (i=0; i<n; i++)
{
    if (a[v][i] == 1)
    {
        indegree[i]--;
        if (indegree[i] == 0)
            queue[rear++] = i;
    }
}
if (count != n)
    printf("in cycle exists");
return 0;
}

```

INPUT:-

enter number of vertices: 6

enter adjacency matrix:-

0	1	1	0	0	0
0	0	0	1	0	0
0	0	0	1	1	0
0	0	0	0	0	1
0	0	0	0	0	1
0	0	0	0	0	0

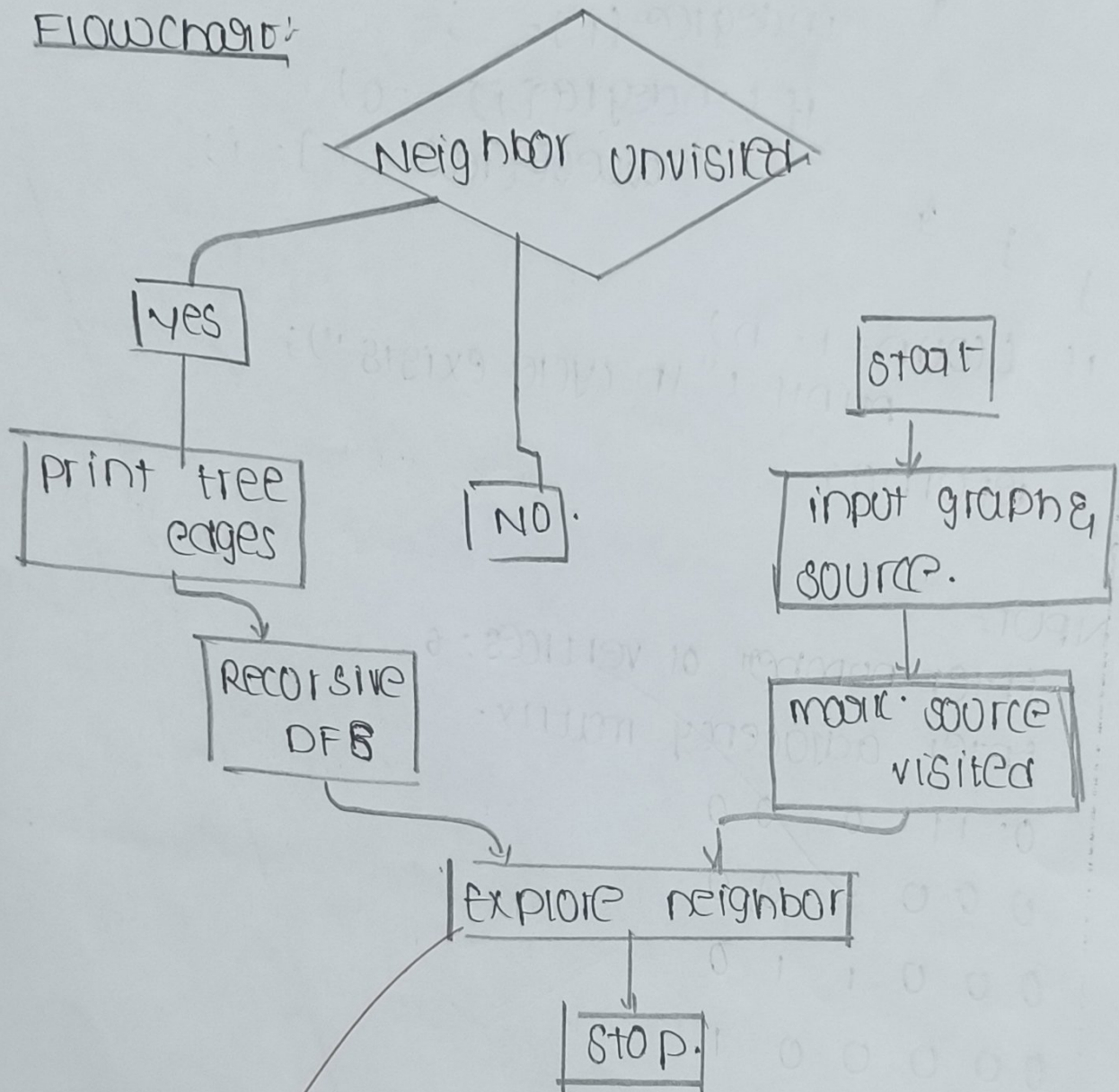
OUTPUT:-

topological sort: 0 1 2 3 4 5.

(H) convert the flowchart for generating a DFS tree into code:

(A)

Flowchart:



Ans

DFS tree - 81

```
#include <stdio.h>
int graph[10][10], visited[10];
int n;
void DFS (int v)
{
    int i;
    for (i=0; i<n; i++)
    {
        if (graph[v][i] == 1)
        {
            if (visited[i] == 0)
            {
                printf ("%d -> %d\n", v, i);
                visited[i] = 1;
                DFS(i);
            }
        }
    }
}

int main()
{
    int source, i, j;
    printf ("enter number of vertices: ");
    scanf ("%d", &n);
    printf ("enter adjacency matrix: \n");
```



```

for (i=0; i<n; i++)
    for (j=0; j<n; j++)
        scanf("%d", &graph[i][j]);
printf ("Enter source vertex:");
scanf ("%d", &source);
visited [source]=1;
printf ("IN DFS TREE edges:\n");
DFS(source);
return 0;
}

```

INPUT:-

Enter number of vertices: 5

Enter adjacency matrix:-

0	1	0	0	0
1	0	0	1	0
1	0	0	0	1
0	✓	0	0	1
0	0	1	1	0

Enter source vertex: 0:

OUTPUT:-

DFS Tree edges:

0 → 1

1 → 3

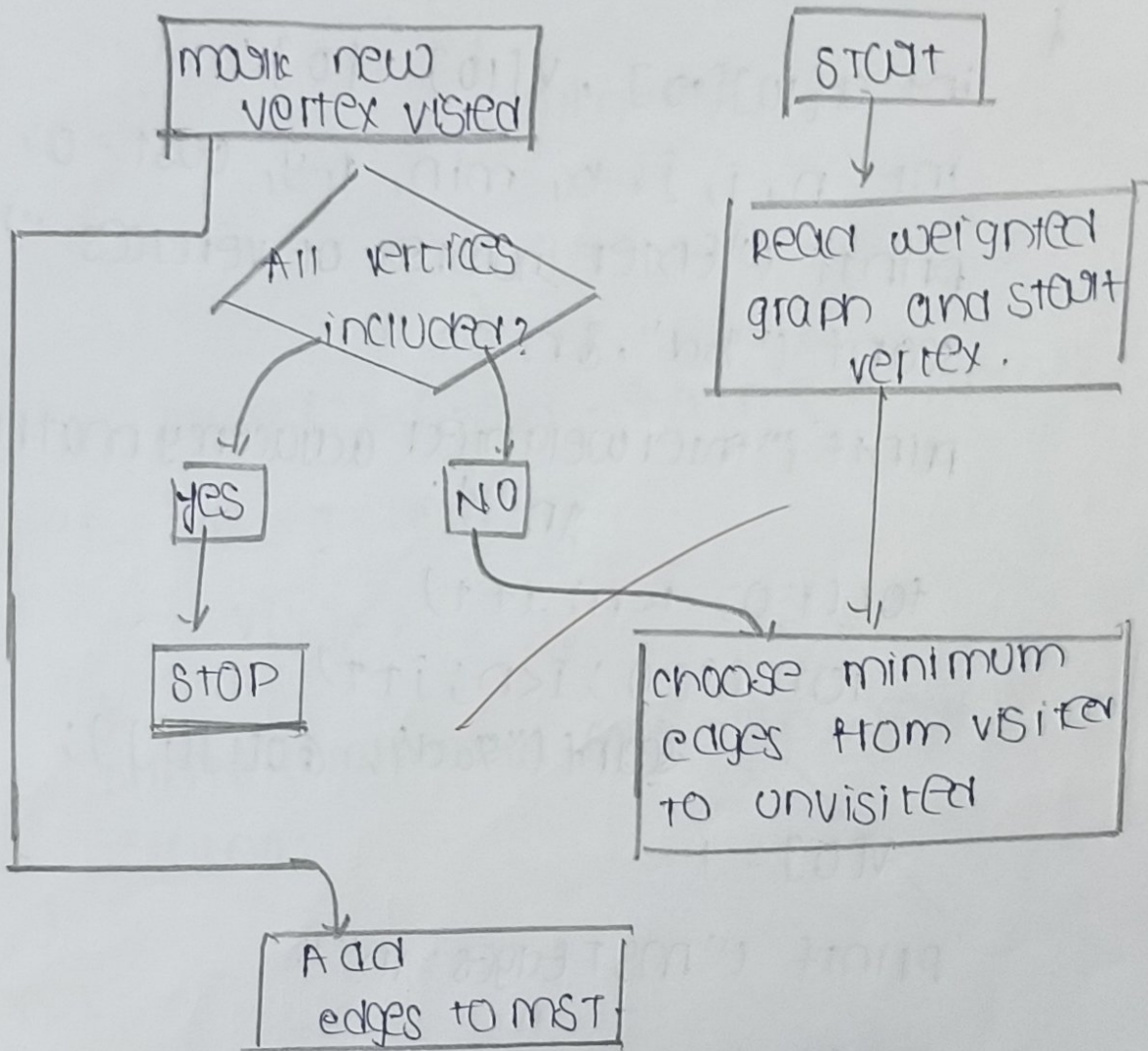
3 → 4

4 → 2

(b) Convert the flowchart for Prim's algorithm into code.

(A)

Flowchart Diagram:



prim's Algorithm.

```
(A) #include <stdio.h>
```

```
int main ( )
```

```
{
```

```
int a[10][10], v[10] = {0};
```

```
int n, i, j, k, min, x, y, cost = 0;
```

```
printf ("Enter number of vertices: ");
```

```
scanf ("%d", &n);
```

```
printf ("Enter weighted adjacency matrix:\n");
```

```
for (i=0; i<n; i++)
```

```
for (j=0; j<n; j++)
```

```
scanf ("%d", &a[i][j]);
```

```
v[0] = 1
```

```
printf ("MST Edges: \n");
```

```
for (k=0; k<n-1; k++)
```

```
{ min = 9999;
```



```
for (i=0; i<n; i++)
```

```
if (v[i])
```

```
for (j=0; j<n; j++)
```

```
if (!v[j] && a[i][j] && a[i][j]<min)
```

```
{ min=a[i][j];
```

```
  x=i;
```

```
  y=j;
```

```
}
```

```
printf("%d - %d = %d\n", x, y, min);
```

```
cost += min;
```

```
v[y] = 1;
```

```
} printf("minimum cost = %d", cost);
```

```
return 0;
```

```
}
```

Input:

4

0 10 6 5

10 0 0 15

6 0 0 4

5 15 4 0

OUTPUT:-

MST edges:

$$0-3 = 5$$

$$3-2 = 4$$

$$0-1 = 10$$

minimum cost = 19.

Wong